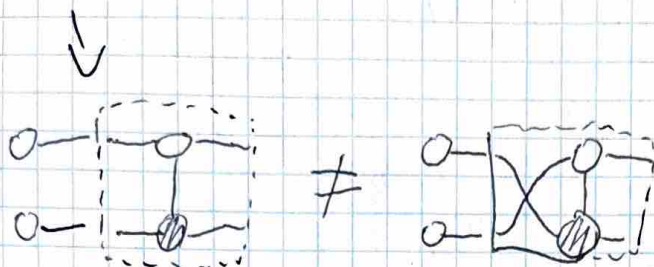
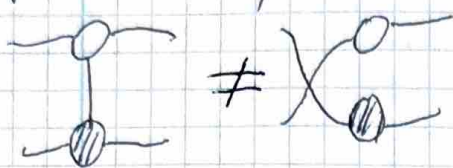
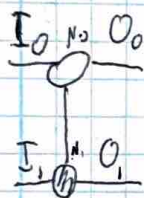


Why hypergraphs?

In general OCM, but when looking at partial diagrams we place pseudo-neg on wire positions



So the digital representation has to keep track of I/O wires



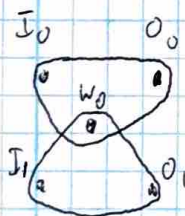
Nodes: $(\mathbb{N}, \mathbb{Z}), (\mathbb{N}, \mathbb{X})$

Edges: $(\mathbb{N}, \mathbb{N})$

Inputs: $(I_0, N_0), (I_1, N_1)$

Outputs: $(O_0, N_0), (O_1, N_1)$

Here we need labels for our nodes & edges



Nodes: (I_0, I_1, w, O_0, O_1)

Edges: $((\mathbb{Z}, \{I_0, O_0, w\}), (\mathbb{X}, \{I_1, O_1, w\}))$

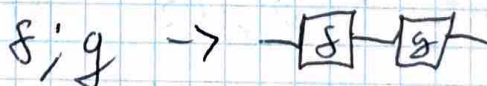
Inputs: (I_0, I_1)

Outputs: (O_0, O_1)

Here our edges (spiders) are anonymous objects

w/ Properties $(\mathbb{Z}/\mathbb{X}/H + \text{phase})$

This structure also better captures how a spider essentially ~~gives~~ connects legs of different tensors



Composition of SMC terms requires identifying output wires of f w/ input wires of g (we can ignore spiders to spiders there always a wire)
So wires should be the first class object

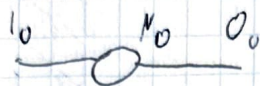
the case

2

(port) graph $\rightarrow$ nodes 1st class objects

$\therefore$ to represent ~~some~~ identity we need to put nodes somewhere

option 1: identity is node



Nodes: (N_0)

Edges: $()$

Inputs: $((I_0, N_0))$

Outputs: $(O_0: N_0)$

option 2: I/O are nodes



Nodes: (I_0, O_0)

Edges: $((I_0, O_0))$

If we then compose

$(id) \times (x_{gate})$

$(x_{gate}) \times id$

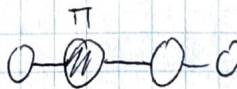
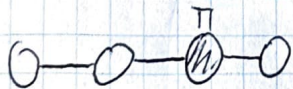
opt 1:

opt 1:



gluing edges

opt 2



gluing nodes

Hypergraph: $\rightarrow$ edges 1st class objects

$id = \bullet$

$x_{gate} = \text{circle with two dots and } \pi$

$(id) \times (x_{gate}) = (x_{gate}) \times (id) = \text{circle with two dots and } \pi = x_{gate}$

gluing nodes
(which are wires)

Hypergraphs

3

hyperedges:

connect arbitrary # of nodes

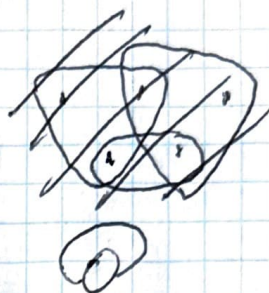
Specific case of hyper-graphs where

hyper edges have:

- ordered list of source nodes
- " " " " target "

Acylicity

no path from a node to itself

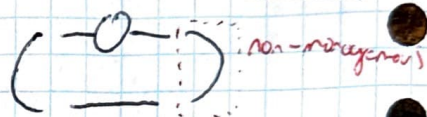
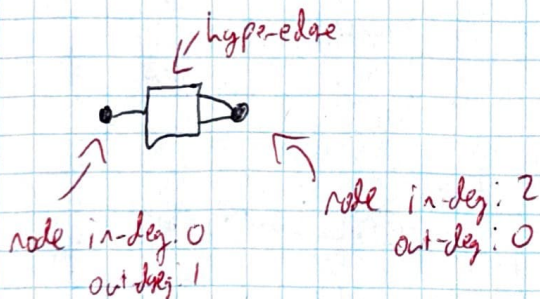


monogamy

degree of node v

in-degree: # of edges with v as target

out-degree: " " " " " " " " source



$$(cup) \times (\cdot \otimes id) \times (cap)$$

Cyclic

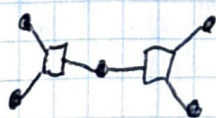
monogamy

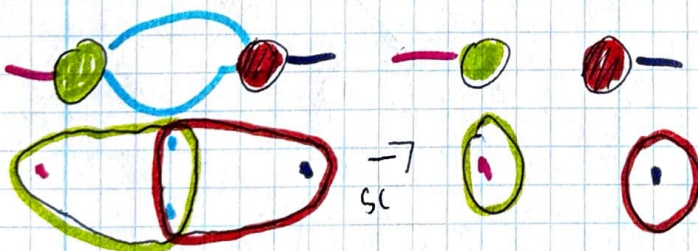
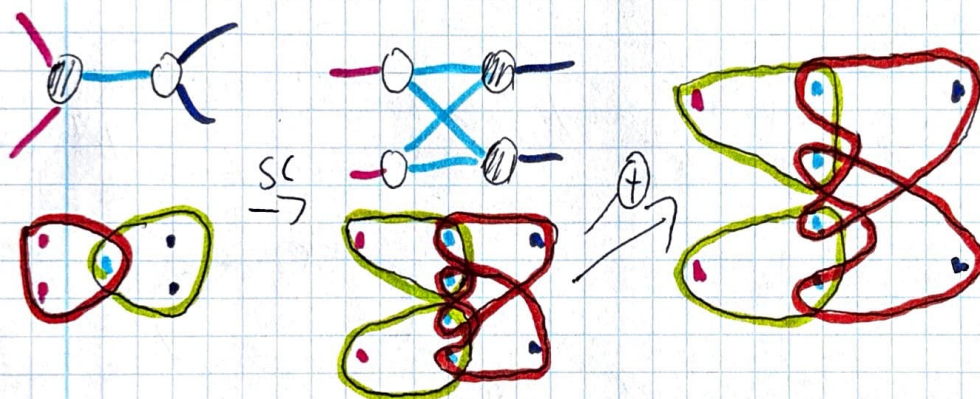
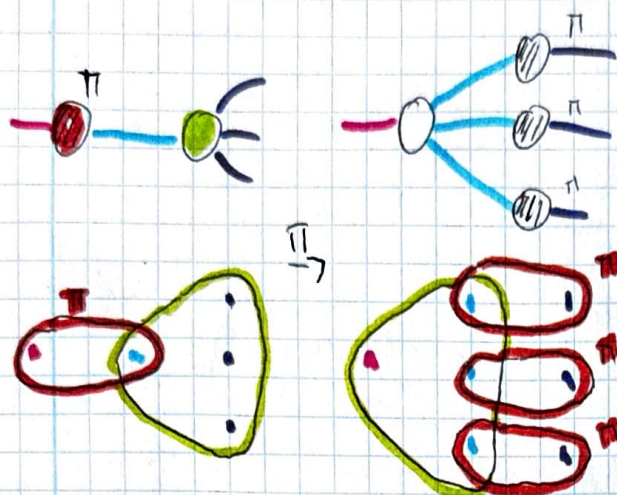
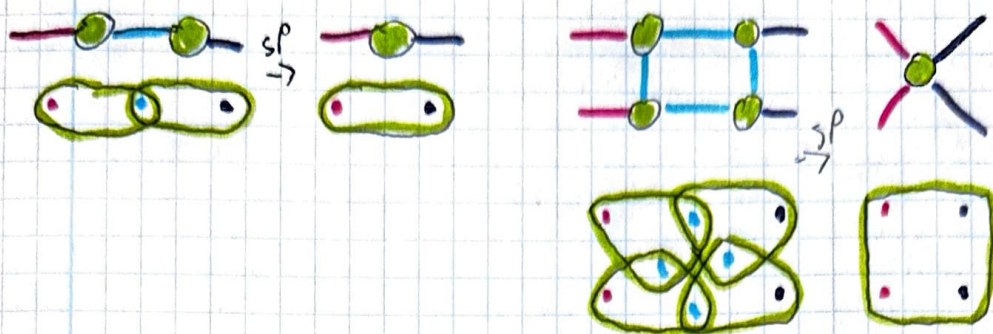
$$\forall v \in G$$

$$indeg(v) = \begin{cases} 0 & \text{if } v \in inputs(G) \\ 1 & \text{otherwise} \end{cases}$$

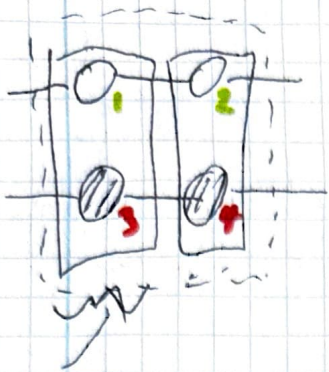
$$outdeg(v) = \begin{cases} 0 & \text{if } v \in outputs(G) \\ 1 & \text{otherwise} \end{cases}$$

This allows a better visualisation





SP + hops $\rightarrow$ edge overlaps w/ even parity disconnect
 " " " odd " connect



$$\{ = d_1 \times d_1 = D_1$$

5/

$$d_1 = (.spider, z \ 11) \& (.spider, x \ 11)$$

wild_rw_chs **SP**

↳ term_rw_helper



$$\{ = (.spider, z \ 11) \times (.spider, z \ 11) = d_2$$



$$\{ = (.spider, x \ 11) \times (.spider, x \ 11) = d_2'$$

return $D_2 = d_2 \& d_2'$

↳ graph_iso_partial_test



return $H_1 \approx_{iso} H_2$

↳ AProp_graph_semantics_Correct

$$AProp \rightarrow Tensor = AProp \rightarrow Hypergraph \rightarrow Tensor$$

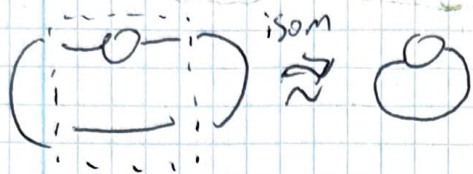
↳ rewrite **SP**

return $\{ = (.spider, z \ 11) \times wire = d_3$ $\} = d_3 \& d_2'$

Tensorflow complications

6/

Cups & Caps cause non-monogamy/cyclicity

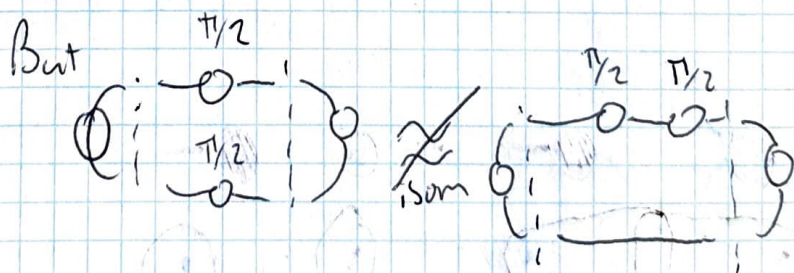
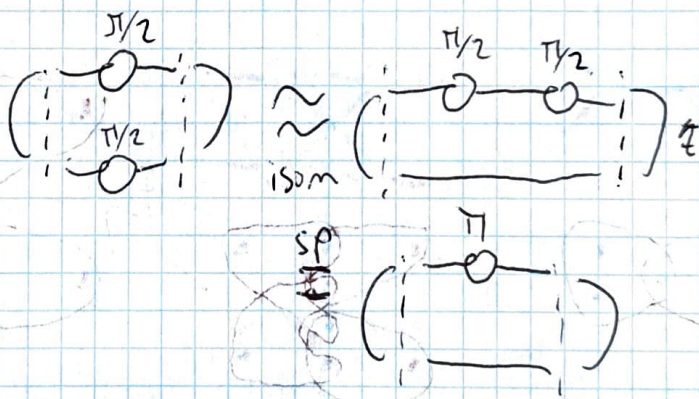


Solution



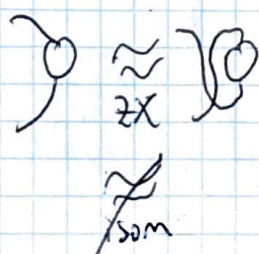
This limits rw abilities

e.g.



So we have to manually manipulate and yanking

Also $\mathbb{Z}/x$ Spider inputs are symmetric



This is because Tensorflow is more general than $\mathbb{Z}x$

Potential to resolve this in LeanSpider